Assignment 1

SET A

Q1) Write a python program to check whether the string is Symmetrical or Palindrome

In [1]:
```
s = input("Enter a string: ")

# check palindrome
pal = (s == s[::-1])

# check symmetrical
mid = len(s)//2
sym = (s[:mid] == s[mid:])

# OR condition
if pal or sym:
    print("String is Palindrome OR Symmetrical")
else:
    print("String is neither")
```

```
String is Palindrome OR Symmetrical
```

Q2) Write a Python program to reverse the words in a given string

In [6]:
```
s = input("Enter a string: ")

words = s.split()
rev = words[::-1]
result = " ".join(rev)

print("Reversed words:", result)
```

```
Reversed words: Morning Good
```

Q3) Write a Python program to remove the i'th character from the string in different ways

Method 1: Using Slicing

In [7]:
```
s = input("Enter string: ")
i = int(input("Enter index to remove: "))

new_s = s[:i] + s[i+1:]
print("After removal:", new_s)
```

```
After removal: Helo
```

Method 2: Using For Loop

In [8]:
```
s = input("Enter string: ")
i = int(input("Enter index to remove: "))

new_s = ""
for index in range(len(s)):
    if index != i:
        new_s += s[index]

print("After removal:", new_s)
```

```
After removal: Helo
```

Method 3: Converting to List

In [10]:
```
s = input("Enter string: ")
i = int(input("Enter index to remove: "))

lst = list(s)
lst.pop(i)
new_s = "".join(lst)

print("After removal:", new_s)
```

```
After removal: Hllo
```

Functions :-

Q1) Write a python function to find the Max of three numbers

In [12]:
```python
def max_of_three(a, b, c):
    if a >= b and a >= c:
        return a
    elif b >= a and b >= c:
        return b
    else:
        return c

x = int(input("Enter first number: "))
y = int(input("Enter second number: "))
z = int(input("Enter third number: "))

print("Maximum number is:", max_of_three(x, y, z))
```

```
Maximum number is: 40
```

Q2) Write a python function to Sum all the numbers in a list

In [13]:
```python
def sum_list(numbers):
    total = 0
    for n in numbers:
        total += n
    return total

lst = [10, 20, 30, 40]

print("Sum of list =", sum_list(lst))
```

```
Sum of list = 100
```

Q3) Write a Python function to reverse the string

In [14]:
```python
def reverse_string(s):
    return s[::-1]

text = input("Enter a string: ")
print("Reversed string:", reverse_string(text))
```

```
Reversed string: dlroW olleH
```

SET B

Strings :--

Q1) Write a python program to print even length words in a string

In [15]:
```
s = input("Enter a string: ")

words = s.split()

print("Even length words:")
for w in words:
    if len(w) % 2 == 0:
        print(w)
```

```
Even length words:
Good
```

Q2) Write a python program to accept the strings which contains all vowels

In [18]:
```
s = input("Enter a string: ")

vowels = "aeiou"
string = s.lower()

flag = True
for v in vowels:
    if v not in string:
        flag = False
        break

if flag:
    print("The string contains all vowels.")
else:
    print("The string does not contain all vowels.")
```

```
The string contains all vowels.
```

Q3) Write a python program to Count the number of matching characters in a pair of String

In [19]:
```python
s1 = input("Enter first string: ")
s2 = input("Enter second string: ")

count = 0

for ch in s1:
    if ch in s2:
        count += 1

print("Number of matching characters:", count)
```

```
Number of matching characters: 3
```

Functions :--

Q1) Write a Python function that takes a list and returns a new list with unique elements of the first list.

In [20]:
```python
def unique_list(lst):
    unique = []
    for item in lst:
        if item not in unique:
            unique.append(item)
    return unique

# Example
numbers = [1, 2, 2, 3, 4, 4, 5]
print("Unique elements:", unique_list(numbers))
```

```
Unique elements: [1, 2, 3, 4, 5]
```

Q2) Write a python function that takes a number as a parameter and check the number is prime or not.

In [23]:
```python
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

```
# Example
num = int(input("Enter a number: "))

if is_prime(num):
    print(num, "is a prime number.")
else:
    print(num, "is not a prime number.")
```

```
5 is a prime number.
```

Q3) Write a python function to check whether a number is perfect or not.

In [26]:
```
def is_perfect(n):
    sum_of_divisors = 0
    for i in range(1, n):
        if n % i == 0:
            sum_of_divisors += i
    return sum_of_divisors == n

# Example
num = int(input("Enter a number: "))

if is_perfect(num):
    print(num, "is a perfect number.")
else:
    print(num, "is not a perfect number.")
```

```
6 is a perfect number.
```

Assignment 2

SET A

1. Python Program to Calculate the Area of a Triangle

In [27]:
```
base = float(input("Enter the base of the triangle: "))
height = float(input("Enter the height of the triangle: "))
```

```
area = 0.5 * base * height

print("Area of the triangle is:", area)
```

```
Area of the triangle is: 14.0
```

2. Python Program to Swap Two Variables

a = input("Enter first variable: ") b = input("Enter second variable: ")

temp = a a = b b = temp

print("After swapping:") print("a =", a) print("b =", b)

3. Python Program to Generate a Random Number

In [29]:
```
import random

start = int(input("Enter the start of range: "))
end = int(input("Enter the end of range: "))

rand_num = random.randint(start, end)

print("Random number:", rand_num)
```

```
Random number: 21
```

SET B

1. Write a Python Program to Check if a Number is Positive, Negative or Zero

In [30]:
```
num = float(input("Enter a number: "))

if num > 0:
    print("The number is Positive.")
elif num < 0:
    print("The number is Negative.")
```

```
else:
    print("The number is Zero.")
```

```
The number is Positive.
```

2. Write a Python Program to Check if a Number is Odd or Even

In [31]:
```
num = int(input("Enter a number: "))

if num % 2 == 0:
    print("The number is Even.")
else:
    print("The number is Odd.")
```

```
The number is Even.
```

3. Write a Python Program to Check Prime Number

In [33]:
```
num = int(input("Enter a number: "))

if num <= 1:
    print(num, "is not a prime number.")
else:
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            print(num, "is not a prime number.")
            break
    else:
        print(num, "is a prime number.")
```

```
17 is a prime number.
```

4. Write a Python Program to Check Armstrong Number

In [35]:
```
num = int(input("Enter a number: "))
sum_of_powers = 0
temp = num
digits = len(str(num))

while temp > 0:
    digit = temp % 10
```

```
    sum_of_powers += digit ** digits
    temp //= 10

if num == sum_of_powers:
    print(num, "is an Armstrong number.")
else:
    print(num, "is not an Armstrong number.")
```

```
153 is an Armstrong number.
```

5. Write a Python Program to Find the Factorial of a Number

In [36]:
```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n-1)

num = int(input("Enter a number: "))

if num < 0:
    print("Factorial does not exist for negative numbers.")
else:
    print("Factorial of", num, "is", factorial(num))
```

```
Factorial of 5 is 120
```

PROGRAMS FOR PRACTICE:

1. Python Program to Convert Kilometers to Miles

In [37]:
```
kilometers = float(input("Enter distance in kilometers: "))

miles = kilometers * 0.621371

print(f"{kilometers} kilometers is equal to {miles} miles.")
```

```
24.0 kilometers is equal to 14.912904000000001 miles.
```

2. Python Program to Convert Celsius To Fahrenheit

In [38]:
```
celsius = float(input("Enter temperature in Celsius: "))

fahrenheit = (celsius * 9/5) + 32

print(f"{celsius}°C is equal to {fahrenheit}°F.")
```

```
20.0°C is equal to 68.0°F.
```

3. Write a Python Program to Check Leap Year

In [40]:
```
year = int(input("Enter a year: "))

if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(year, "is a leap year.")
else:
    print(year, "is not a leap year.")
```

```
2012 is a leap year.
```

4. Write a Python Program to Print all Prime Numbers in an Interval

In [41]:
```
start = int(input("Enter the start of interval: "))
end = int(input("Enter the end of interval: "))

print("Prime numbers between", start, "and", end, "are:")

for num in range(start, end + 1):
    if num > 1:
        for i in range(2, int(num ** 0.5) + 1):
            if num % i == 0:
                break
        else:
            print(num, end=" ")
```

```
Prime numbers between 10 and 20 are:
11 13 17 19
```

5. Write a Python Program to Print the Fibonacci sequence

In [42]:
```python
n_terms = int(input("Enter the number of terms: "))

a, b = 0, 1
count = 0

print("Fibonacci sequence:")

while count < n_terms:
    print(a, end=" ")
    a, b = b, a + b
    count += 1
```

```
Fibonacci sequence:
0 1 1 2 3 5 8 13 21 34 55 89 144
```

6. Write a Python Program to Find Armstrong Number in an Interval

In [44]:
```python
start = int(input("Enter the start of interval: "))
end = int(input("Enter the end of interval: "))

print(f"Armstrong numbers between {start} and {end} are:")

for num in range(start, end + 1):
    sum_of_powers = 0
    temp = num
    digits = len(str(num))

    while temp > 0:
        digit = temp % 10
        sum_of_powers += digit ** digits
        temp //= 10

    if num == sum_of_powers:
        print(num, end=" ")
```

```
Armstrong numbers between 100 and 500 are:
153 370 371 407
```

7. Write a Python Program to Find the Sum of Natural Numbers

In [45]:
```
n = int(input("Enter a positive integer: "))

sum_natural = 0
for i in range(1, n + 1):
    sum_natural += i

print("The sum of first", n, "natural numbers is:", sum_natural)
```

```
The sum of first 20 natural numbers is: 210
```

Assignment 3

1. Write a Python program to append items from a specified list.

In [46]:
```
list1 = [1, 2, 3]
print("Original list1:", list1)

list2 = [4, 5, 6]
print("List to append:", list2)

for item in list2:
    list1.append(item)

print("List after appending items:", list1)
```

```
Original list1: [1, 2, 3]
List to append: [4, 5, 6]
List after appending items: [1, 2, 3, 4, 5, 6]
```

2. Write a python program Check if a Substring is Present in a Given String

In [48]:
```
text = input("Enter the main string: ")

substring = input("Enter the substring to check: ")

if substring in text:
    print(f"'{substring}' is present in the string.")
else:
    print(f"'{substring}' is not present in the string.")
```

```
'World' is present in the string.
```

3. Write a python program Words Frequency in String Shorthands

In [50]:
```
text = input("Enter a string: ")

freq = {}
for word in text.split():
    freq[word] = freq.get(word, 0) + 1

print("Word Frequency:")
for word, count in freq.items():
    print(f"{word}: {count}")
```

```
Word Frequency:
Hello: 1
```

4. Write a python program Convert Snake case to Pascal case

In [52]:
```
snake_str = input("Enter a snake_case string: ")

pascal_str = ''.join(word.capitalize() for word in snake_str.split('_'))

print("PascalCase string:", pascal_str)
```

```
PascalCase string: HelloWorldPython
```

5. Write a Python function to calculate the factorial of a number (a non-negative integer). The function accepts the number as an argument.

In [53]:
```
def factorial(n):
    if n < 0:
```

```
        return "Factorial not defined for negative numbers"

    fact = 1
    for i in range(1, n + 1):
        fact *= i

    return fact

num = int(input("Enter a non-negative number: "))

print("Factorial of", num, "is:", factorial(num))
```

```
Factorial of 5 is: 120
```

6. Write a Python function to check whether a number is in a given range.

In [55]:
```
def check_range(num, start, end):
    if start <= num <= end:
        return True
    else:
        return False

num = int(input("Enter a number: "))
start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

if check_range(num, start, end):
    print(num, "is in the range.")
else:
    print(num, "is NOT in the range.")
```

```
20 is in the range.
```

7. Write a Python function that accepts a string and calculate the number of upper case letters and lower case letters.

In [56]:
```
def count_upper_lower(s):
    upper = 0
    lower = 0

    for ch in s:
        if ch.isupper():
```

```
            upper += 1
        elif ch.islower():
            lower += 1

    return upper, lower

text = input("Enter a string: ")

u, l = count_upper_lower(text)

print("Uppercase letters:", u)
print("Lowercase letters:", l)
```

```
Uppercase letters: 4
Lowercase letters: 12
```

8. Write a Python program to detect the number of local variables declared in a function.

In [57]:
```
def sample_function():
    x = 10
    y = 20
    z = x + y
    name = "Python"

# Count local variables
print("Number of local variables:", sample_function.__code__.co_nlocals)
```

```
Number of local variables: 4
```

8. Write a python program to Remove all duplicates from a given string in Python

In [58]:
```
string = input("Enter a string: ")

result = ""
for ch in string:
    if ch not in result:
        result += ch

print("String after removing duplicates:", result)
```

```
String after removing duplicates: Helo
```

9. Write a Python function that checks whether a passed string is palindrome or not.

In [59]:
```
def is_palindrome(s):
    s = s.lower()
    if s == s[::-1]:
        return True
    else:
        return False

text = input("Enter a string: ")

if is_palindrome(text):
    print("The string is a palindrome.")
else:
    print("The string is NOT a palindrome.")
```

```
The string is a palindrome.
```

11. Write a Python program that accepts a hyphen-separated sequence of words as input and prints the words in a hyphen-separated sequence after sorting them alphabetically.

In [60]:
```
words = input("Enter hyphen-separated words: ")

sorted_words = "-".join(sorted(words.split("-")))

print("Sorted sequence:", sorted_words)
```

```
Sorted sequence: blue-green-red-yellow
```

Assignment 4

SET A

List

1) Write a Python program to sum all the items in a list.

In [61]:
```
numbers = [10, 20, 30, 40, 50]

total = 0
for num in numbers:
    total += num

print("Sum of all items in the list:", total)
```

```
Sum of all items in the list: 150
```

2) Write a Python program to multiplies all the items in a list.

In [62]:
```
numbers = [2, 3, 4, 5]

product = 1
for num in numbers:
    product *= num

print("Product of all items in the list:", product)
```

```
Product of all items in the list: 120
```

3) Write a Python program to get a list, sorted in increasing order by the last element in each tuple from a given list of non-empty tuples.

In [1]:
```
def last_element(t):
    return t[-1]

tuples_list = [(2, 5), (1, 2), (4, 4), (2, 3), (9, 1)]

sorted_list = sorted(tuples_list, key=last_element)

print("Sorted list:", sorted_list)
```

```
Sorted list: [(9, 1), (1, 2), (2, 3), (4, 4), (2, 5)]
```

Tuples:--

1) Write a Python program to create a tuple.

In [1]:
```python
my_tuple = (10, 20, 30, 40, 50)

print("The created tuple is:", my_tuple)
```

```
The created tuple is: (10, 20, 30, 40, 50)
```

2) Write a Python program to create a tuple with different data types.

In [2]:
```python
my_tuple = (25, "Hello", 3.14, True, [1, 2, 3])

print("Tuple with different data types:", my_tuple)
```

```
Tuple with different data types: (25, 'Hello', 3.14, True, [1, 2, 3])
```

3) Write a Python program to check whether an element exists within a tuple.

In [3]:
```python
my_tuple = (10, 20, 30, 40, 50)

element = 30

if element in my_tuple:
    print(f"{element} exists in the tuple.")
else:
    print(f"{element} does not exist in the tuple.")
```

```
30 exists in the tuple.
```

Sets:--

1) Write a Python program to create a set.

In [4]:
```python
my_set = {10, 20, 30, 40, 50}

print("The created set is:", my_set)
```

```
The created set is: {50, 20, 40, 10, 30}
```

2) Write a Python program to iterate over sets.

In [5]:
```
my_set = {"apple", "banana", "cherry", "mango"}

for item in my_set:
    print(item)
```

```
apple
banana
cherry
mango
```

4) Write a Python program to create set difference.

In [6]:
```
set1 = {10, 20, 30, 40, 50}
set2 = {30, 40, 60, 70}

difference = set1 - set2

print("Set difference (set1 - set2):", difference)
```

```
Set difference (set1 - set2): {50, 10, 20}
```

Dictionary:--

1) Write a Python script to sort (ascending and descending) a dictionary by value.

In [7]:
```
my_dict = {'a': 45, 'b': 12, 'c': 78, 'd': 34}

ascending = dict(sorted(my_dict.items(), key=lambda item: item[1]))

descending = dict(sorted(my_dict.items(), key=lambda item: item[1], reverse=True))

print("Ascending order:", ascending)
print("Descending order:", descending)
```

```
Ascending order: {'b': 12, 'd': 34, 'a': 45, 'c': 78}
Descending order: {'c': 78, 'a': 45, 'd': 34, 'b': 12}
```

2) Write a Python script to add a key to a dictionary.

In [8]:
```python
my_dict = {1: 'Apple', 2: 'Banana'}

# Adding a new key-value pair
my_dict[3] = 'Cherry'

print("Updated dictionary:", my_dict)
```

```
Updated dictionary: {1: 'Apple', 2: 'Banana', 3: 'Cherry'}
```

3) Write a Python program to iterate over dictionaries using for loops.

In [9]:
```python
my_dict = {'name': 'Alice', 'age': 25, 'city': 'Mumbai'}

# Iterating over keys
print("Iterating over keys:")
for key in my_dict:
    print(key)

# Iterating over values
print("\nIterating over values:")
for value in my_dict.values():
    print(value)

# Iterating over key-value pairs
print("\nIterating over key-value pairs:")
for key, value in my_dict.items():
    print(key, ":", value)
```

```
Iterating over keys:
name
age
city

Iterating over values:
Alice
25
Mumbai

Iterating over key-value pairs:
name : Alice
age : 25
city : Mumbai
```

SET B

List

1. Write a Python program to remove duplicates from a list.

In [10]:
```python
my_list = [10, 20, 10, 30, 20, 40, 30]

# Removing duplicates by converting to a set
unique_list = list(set(my_list))

print("List after removing duplicates:", unique_list)
```

```
List after removing duplicates: [40, 10, 20, 30]
```

2. Write a Python program to check a list is empty or not.

In [11]:
```python
my_list = []

if not my_list:
    print("The list is empty.")
else:
    print("The list is not empty.")
```

```
The list is empty.
```

Tuples

1. Write a Python program to convert a list to a tuple.

In [12]:
```
my_list = [10, 20, 30, 40, 50]

my_tuple = tuple(my_list)

print("List:", my_list)
print("Converted Tuple:", my_tuple)
```

```
List: [10, 20, 30, 40, 50]
Converted Tuple: (10, 20, 30, 40, 50)
```

2. Write a Python program to remove an item from a tuple.

In [13]:
```
my_tuple = (10, 20, 30, 40, 50)

item_to_remove = 30

# Converting tuple to list to remove the item
temp_list = list(my_tuple)
temp_list.remove(item_to_remove)

new_tuple = tuple(temp_list)

print("Original Tuple:", my_tuple)
print("Tuple after removing item:", new_tuple)
```

```
Original Tuple: (10, 20, 30, 40, 50)
Tuple after removing item: (10, 20, 40, 50)
```

3. Write a Python program to slice a tuple.

In [14]:
```
my_tuple = (10, 20, 30, 40, 50, 60, 70)

slice1 = my_tuple[1:4]
slice2 = my_tuple[:3]
slice3 = my_tuple[3:]
slice4 = my_tuple[-4:-1]
```

```
print("Original Tuple:", my_tuple)
print("Slice 1 (1:4):", slice1)
print("Slice 2 (:3):", slice2)
print("Slice 3 (3:):", slice3)
print("Slice 4 (-4:-1):", slice4)
```

```
Original Tuple: (10, 20, 30, 40, 50, 60, 70)
Slice 1 (1:4): (20, 30, 40)
Slice 2 (:3): (10, 20, 30)
Slice 3 (3:): (40, 50, 60, 70)
Slice 4 (-4:-1): (40, 50, 60)
```

4. Write a Python program to find the length of a tuple.

In [15]:
```
my_tuple = (10, 20, 30, 40, 50)

length = len(my_tuple)

print("Length of the tuple:", length)
```

```
Length of the tuple: 5
```

Dictionary

1. Write a Python script to generate and print a dictionary that contains a number (between 1 and n) in the form (x, x*x).

In [16]:
```
n = int(input("Enter a number: "))

result = {}

for x in range(1, n + 1):
    result[x] = x * x

print("Generated dictionary:", result)
```

```
Generated dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81, 10: 100, 11: 121, 12: 144}
```

3. Write a Python script to merge two Python dictionaries.

In [17]:
```python
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}

# Method 1: Using update()
merged_dict = dict1.copy()  # To keep dict1 unchanged
merged_dict.update(dict2)

print("Merged dictionary:", merged_dict)

# Method 2: Using dictionary unpacking
merged_dict2 = {**dict1, **dict2}
print("Merged dictionary (using unpacking):", merged_dict2)
```

```
Merged dictionary: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
Merged dictionary (using unpacking): {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

4. Write a Python program to get a dictionary from an object's fields.

In [18]:
```python
# Define a class
class Person:
    def __init__(self, name, age, city):
        self.name = name
        self.age = age
        self.city = city

# Create an object
person1 = Person("Alice", 25, "Mumbai")

# Get dictionary from object's fields
person_dict = person1.__dict__

print("Dictionary from object's fields:", person_dict)
```

```
Dictionary from object's fields: {'name': 'Alice', 'age': 25, 'city': 'Mumbai'}
```

PROGRAMS FOR PRACTICE:

1. Write a Python program to get the largest number from a list.

In [1]:
```python
numbers = [25, 78, 56, 89, 102, 45]

largest = max(numbers)

print("The largest number is:", largest)
```

```
The largest number is: 102
```

2. Write a Python program to get the smallest number from a list.

In [2]:
```python
numbers = [25, 78, 56, 89, 5, 45]

smallest = min(numbers)

print("The smallest number is:", smallest)
```

```
The smallest number is: 5
```

3. Write a Python program to count the number of strings where the string length is 2 or more and the first and last character are same from a given list of strings.

In [3]:
```python
words = ["abc", "xyz", "aba", "1221", "aa", "bcb", "hello"]

count = 0

for w in words:
    if len(w) >= 2 and w[0] == w[-1]:
        count += 1

print("Number of valid strings:", count)
```

```
Number of valid strings: 4
```

3. Write a Python program to add an item in a tuple.

In [4]:
```python
my_tuple = (1, 2, 3)

temp = list(my_tuple)

# Add item
temp.append(4)
```

```
my_tuple = tuple(temp)

print(my_tuple)
```

```
(1, 2, 3, 4)
```

4. Write a Python program to convert a tuple to a string.

In [5]:
```
t = ('H', 'e', 'l', 'l', 'o')

result = ''.join(t)

print(result)
```

```
Hello
```

5. Write a Python program to create the colon of a tuple.

In [6]:
```
t1 = (1, 2, 3, 4, 5)

# Clone the tuple
t2 = tuple(t1)

print("Original tuple:", t1)
print("Cloned tuple:", t2)
```

```
Original tuple: (1, 2, 3, 4, 5)
Cloned tuple: (1, 2, 3, 4, 5)
```

6. Write a Python program to unpack a tuple in several variables.

In [7]:
```
my_tuple = (10, 20, 30, 40)

# Unpacking the tuple
a, b, c, d = my_tuple

print("a =", a)
print("b =", b)
print("c =", c)
print("d =", d)
```

```
a = 10
b = 20
c = 30
d = 40
```

7. Write a Python program to add member(s) in a set.

In [8]:
```
my_set = {1, 2, 3}

my_set.add(4)

print("Updated set:", my_set)
```

```
Updated set: {1, 2, 3, 4}
```

8. Write a Python program to remove item(s) from set

In [9]:
```
my_set = {1, 2, 3, 4, 5}

my_set.remove(3)

print("Updated set:", my_set)
```

```
Updated set: {1, 2, 4, 5}
```

9. Write a Python program to create an intersection of sets.

In [10]:
```
set1 = {1, 2, 3, 4, 5}
set2 = {4, 5, 6, 7}

result = set1.intersection(set2)

print("Intersection:", result)
```

```
Intersection: {4, 5}
```

11. Write a Python program to create a union of sets.

In [11]:
```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
```

```
result = set1.union(set2)

print("Union of sets:", result)
```

```
Union of sets: {1, 2, 3, 4, 5, 6}
```

12. Write a Python script to concatenate following dictionaries to create a new one.

In [12]:
```
dic1 = {1: 10, 2: 20}
dic2 = {3: 30, 4: 40}
dic3 = {5: 50, 6: 60}

new_dict = {}

for d in (dic1, dic2, dic3):
    new_dict.update(d)

print("Concatenated dictionary:", new_dict)
```

```
Concatenated dictionary: {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}
```

13. Write a Python program to map two lists into a dictionary.

In [13]:
```
keys = ['a', 'b', 'c']
values = [1, 2, 3]

my_dict = {}

for i in range(len(keys)):
    my_dict[keys[i]] = values[i]

print(my_dict)
```

```
{'a': 1, 'b': 2, 'c': 3}
```

14. Write a Python program to sort a dictionary by key.

In [14]:
```
my_dict = {3: "Apple", 1: "Banana", 2: "Cherry"}

# Sorting dictionary by key
sorted_dict = dict(sorted(my_dict.items()))
```

```
print("Dictionary sorted by keys:", sorted_dict)
```

```
Dictionary sorted by keys: {1: 'Banana', 2: 'Cherry', 3: 'Apple'}
```

15. Write a Python program to get the maximum and minimum value in a dictionary.

In [15]:
```
my_dict = {'a': 15, 'b': 8, 'c': 27, 'd': 3}

# Maximum value
max_value = max(my_dict.values())

# Minimum value
min_value = min(my_dict.values())

print("Maximum value in the dictionary:", max_value)
print("Minimum value in the dictionary:", min_value)
```

```
Maximum value in the dictionary: 27
Minimum value in the dictionary: 3
```

16. Write a Python program to clone or copy a list.

In [16]:
```
my_list = [10, 20, 30]

cloned_list = my_list[:]

print("Cloned List:", cloned_list)
```

```
Cloned List: [10, 20, 30]
```

17. Write a Python program to find the list of words that are longer than n from a given list of words.

In [18]:
```
words = ["Python", "is", "awesome", "and", "powerful"]

n = 5

long_words = [word for word in words if len(word) > n]

print("Words longer than", n, "characters:", long_words)
```

```
Words longer than 5 characters: ['Python', 'awesome', 'powerful']
```

18. Write a Python program to unzip a list of tuples into individual lists.

In [19]:
```python
tuple_list = [(1, 'a'), (2, 'b'), (3, 'c')]

# Unzipping using zip(*)
numbers, letters = zip(*tuple_list)

# Converting tuples to lists
numbers = list(numbers)
letters = list(letters)

print("Numbers:", numbers)
print("Letters:", letters)
```

```
Numbers: [1, 2, 3]
Letters: ['a', 'b', 'c']
```

19. Write a Python program to reverse a tuple.

In [20]:
```python
my_tuple = (1, 2, 3, 4, 5)

# Reversing the tuple using slicing
reversed_tuple = my_tuple[::-1]

print("Original tuple:", my_tuple)
print("Reversed tuple:", reversed_tuple)
```

```
Original tuple: (1, 2, 3, 4, 5)
Reversed tuple: (5, 4, 3, 2, 1)
```

20. Write a Python program to convert a list of tuples into a dictionary.

In [21]:
```python
tuple_list = [('a', 1), ('b', 2), ('c', 3)]

# Converting to dictionary
my_dict = dict(tuple_list)

print("Converted dictionary:", my_dict)
```

```
Converted dictionary: {'a': 1, 'b': 2, 'c': 3}
```

21. Write a Python program to print a tuple with string formatting.

In [22]:
```python
my_tuple = ('Alice', 25, 'Engineer')

print(f"Name: {my_tuple[0]}, Age: {my_tuple[1]}, Profession: {my_tuple[2]}")
```

```
Name: Alice, Age: 25, Profession: Engineer
```

22. Write a Python program to create a symmetric difference.

In [23]:
```python
set1 = {'a', 'b', 'c'}
set2 = {'b', 'c', 'd'}

sym_diff = set1 ^ set2

print("Symmetric difference:", sym_diff)
```

```
Symmetric difference: {'d', 'a'}
```

23. Write a Python program to check if a given value is present in a set or not.

In [24]:
```python
def check_value(s, val):
    return val in s

my_set = {'apple', 'banana', 'cherry'}
value = 'banana'

if check_value(my_set, value):
    print(f"{value} is present in the set.")
else:
    print(f"{value} is not present in the set.")
```

```
banana is present in the set.
```

24. Write a Python program to check if a given set is superset of itself and superset of another given set.

In [25]:
```python
set1 = {1, 2, 3, 4, 5}
set2 = {2, 3, 4}
```

```
# Check if set1 is superset of itself
print("Is set1 a superset of itself?", set1.issuperset(set1))

# Check if set1 is superset of set2
print("Is set1 a superset of set2?", set1.issuperset(set2))
```

```
Is set1 a superset of itself? True
Is set1 a superset of set2? True
```

25. Write a Python program to check a given set has no elements in common with other given set.

In [26]:
```
if len(set1 & set2) == 0:
    print("The sets have no elements in common.")
else:
    print("The sets have some common elements.")
```

```
The sets have some common elements.
```

26. Write a Python program to remove the intersection of a 2nd set from the 1st set.

In [27]:
```
set1 = {1, 2, 3, 4, 5}
set2 = {4, 5, 6, 7}

set1 = set1 - set2

print("Set1 after removing intersection:", set1)
```

```
Set1 after removing intersection: {1, 2, 3}
```

27. Write a Python program to remove duplicates from Dictionary.

In [28]:
```
my_dict = {'a': 10, 'b': 20, 'c': 10, 'd': 30, 'e': 20}

# Remove duplicates
seen = set()
new_dict = {}

for key, value in my_dict.items():
    if value not in seen:
        new_dict[key] = value
        seen.add(value)
```

```
print("Dictionary after removing duplicates:", new_dict)
```

```
Dictionary after removing duplicates: {'a': 10, 'b': 20, 'd': 30}
```

28. Write a Python script to check whether a given key already exists in a dictionary.

In [29]:
```
my_dict = {'name': 'Alice', 'age': 25, 'city': 'Mumbai'}

# Key to check
key_to_check = 'age'

if key_to_check in my_dict:
    print(f"The key '{key_to_check}' exists in the dictionary.")
else:
    print(f"The key '{key_to_check}' does not exist in the dictionary.")
```

```
The key 'age' exists in the dictionary.
```

29. Write a Python program to sum all the items in a dictionary.

In [30]:
```
my_dict = {'a': 100, 'b': 200, 'c': 300}

total = sum(my_dict.values())

print("Sum of all items in the dictionary:", total)
```

```
Sum of all items in the dictionary: 600
```

30. Write a Python program to multiply all the items in a dictionary.

In [32]:
```
my_dict = {'a': 2, 'b': 3, 'c': 4}

# Initialize product
product = 1

for value in my_dict.values():
    product *= value

print("Product of all items in the dictionary:", product)
```

```
Product of all items in the dictionary: 24
```

31. Write a Python program to remove a key from a dictionary.

In [34]:
```python
my_dict = {'name': 'Alice', 'age': 25, 'city': 'Mumbai'}

key_to_remove = 'age'

# Remove key using pop()
removed_value = my_dict.pop(key_to_remove, None)

print("Updated dictionary:", my_dict)
print("Removed value:", removed_value)
```

```
Updated dictionary: {'name': 'Alice', 'city': 'Mumbai'}
Removed value: 25
```

In [ ]:
```python

```